Problem B:

Regression Failure Bucketing

Grant Gung-Yu Pan
(Synopsys, Inc.)

Q&A

Q1. I am writing to inquire about the LLM API configurations described in the problem statement.
According to Section 3.4 (LLM Usage & Configuration), the final evaluation will be conducted using two specific endpoints: one for a Qwen model and another for GPT-4o. While the document mentions that contestants may use their own endpoints during the development phase, we would like to know if the organizers provide any official test API endpoints.
Could you please clarify if such testing resources are available to contestants before the alpha submission?

A1. No, we cannot provide LLM endpoint outside Synopsys.
This is the nature of GenAI applications in EDA industry.
Build application with development endpoint, ship to customer, and use with deployment endpoint.
Once the model is the same, the behavior should be the same.

Q2. I am working on ICCAD Problem B: Regression Failure Bucketing, and I would like to clarify the expected granularity of the injected bugs/faults.

From the problem statement, each golden bucket corresponds to one injected bug, and each failure case is assumed to be primarily caused by one injected bug. However, the statement does not specify the granularity or scope of these injected bugs.

Could you clarify whether the injected bugs are limited to certain coarse-grained functional categories or RTL subsystems, such as decoder, debug controller, CSR/privilege logic, branch/PC control, load-store unit, memory interface, etc.? Or should we assume that the injected bugs may occur anywhere in the modified Ibex RTL without a predefined fault taxonomy?

In other words, is it reasonable for contestants to build a diagnosis-inspired feature extractor that maps failure symptoms to coarse suspect regions of the Ibex design, or are the injected bugs intended to be arbitrary RTL modifications where no such coarse category assumption should be made?

I understand that the actual injected bugs and golden buckets cannot be disclosed. I am only asking whether there is any intended coarse-grained fault scope or fault model that participants may rely on when designing the bucketing algorithm.

A2. There is no predefined fault taxonomy or fixed list of coarse subsystems, and different bugs can produce overlapping symptoms across coarse-grained subsystems. In short, feel free to exploit RTL/DV knowledge for feature engineering, but please design the bucketing algorithm to be open-ended over the bug space rather than assume a fixed fault model.

Q3. I would like to ask which version of the Ibex core was used to generate the benchmark cases for this contest. If possible, could you provide the specific tag, branch, or commit hash of the Ibex repository?

A3. You can check out using the hash from the upstream lowRISC Ibex repo:
git clone https://github.com/lowRISC/ibex.git
git checkout 8ce399dbe678f0a66856ac302ec7609ba366d8fd
Note: the RTL was locally modified for bug injection on top of this commit.

Q4. I am working on ICCAD Problem B: Regression Failure Bucketing, and I would like to ask whether the official evaluation environment will provide GPU access for submitted programs. If GPU access is available, could you please let us know the expected GPU model and memory size?

A4. The host machine is not equipped with GPU. If your program uses LLM, it connects to the endpoint with GPU via HTTP.

Q5. A few days ago, you updated the golden files in the test cases. We noticed that in Benchmark 1, the three log files in Case 5 and Case 9 are almost identical, with the only difference being their runtime timestamps. However, in the official golden files,

they are assigned to different buckets. This does not seem to be a sufficient basis for placing them into different buckets.Therefore, we are writing to ask whether there might be an error in the golden files.

A5. Thank you for the careful investigation. The same syndrome you see is actually coming from two different bugs, while failing on the same regression test. Although this is very common in industrial regression bucketing, to reduce the noise from development and evaluation, we have excluded all these conflicting cases from the benchmark sets (and made some swapping for size reasons), both public and private. Please refer to the update benchmark sets.

Q6. We would like to ask about the evaluation/testing environment for the contest, since the runtime limit is important for our implementation. Could you please provide some details about the machine used for evaluation, such as the CPU model, the maximum number of CPU cores/threads we are allowed to use, and the available memory size?

A6. The CPU has 32 logical cores available to your process with ~128GB RAM. For Alpha/Beta tests, we will enlarge the timeout threshold by 3X/2X and report the actual runtime, so you can optimize your program accordingly.

Q7. Does the modified source code belong to the CPU design or the simulation tool? In other words, are we trying to detect a bug in the design or in the tool? Does the assembly language in the trace file represent the execution process of the simulation tool or the CPU design?

A7. The bugs are injected into the RTL source code where the assembly trace is dumped from simulation of the CPU design. The simulator (VCS) and testbench (UVM) are oracle.

Q8. Could you please let us know which ISS version you are using? Also, is the UVM version based on the Ibx version?

A8. Ibex uses Spike as a golden reference. UVM version is determined by the simulator where you can find in sim.log.gz: UVM-1.2

Q9. According to the latest announcement, teams working on Problems B should submit their files through the provided Google Drive links by placing them into the alpha_test_submission folder. In addition, the problem description states that the submitted program should be executed as follows:

regn_fail_bucketing –input –output –k

It also mentions that for programs written in Python, it is strongly recommended to pack the entire environment using PyInstaller, and that uploading the source code is optional. Only if the executable program cannot run, the evaluator will try running the source code instead.

We would like to confirm how the runtime environment and executable compatibility will be handled during evaluation. Since a PyInstaller executable is usually not guaranteed to run on every machine, it may still depend on the operating system, CPU architecture, glibc version, and native libraries used by Python packages.

In particular, we would like to ask the following questions:

1. What is the official evaluation environment for Problems B and C?

- What operating system and version will be used?
- What CPU architecture will be used?
- Is there any information about the glibc version or Linux distribution?

2. For Python submissions using PyInstaller:

- Should the executable be built on a specific Linux version to ensure compatibility?
- Is the expected executable name exactly regn_fail_bucketing?
- Should we submit only the PyInstaller executable, or should we also include the source code and dependency files as a fallback?
- If our Python program uses packages such as NumPy, pandas, scikit-learn, or other packages with native libraries, is PyInstaller packaging sufficient?
- If the executable fails due to environment compatibility, will the evaluator automatically try to run the submitted source code?

3. For Python source-code fallback:

- What Python version will be available in the evaluation environment?

- Are we allowed to submit requirements.txt, environment.yml, or similar dependency files?
- Will the evaluator install the required Python packages before running the source code?
- Is internet access available during evaluation, or should all dependencies be included in the submission?

4. For C/C++ submissions:

- Should we submit source code with a Makefile/CMakeLists.txt, or should we submit a precompiled executable?
- What compiler and version will be used, such as GCC or Clang?
- Which C++ standard versions are allowed, such as C++17 or C++20?
- If we submit a precompiled executable, how should we avoid library compatibility issues such as glibc or libstdc++ version mismatches?

5. For Docker-based submissions:

- Is Docker allowed for Problems B and C?
- If Docker is allowed, should we submit a Dockerfile, a Docker image archive, or only build scripts?
- Are there any restrictions on Docker image size, base image, runtime permissions, or network access?

6. Before the deadline, is there any recommended way to verify that our submitted executable, source code, dependencies, file structure, and execution command are acceptable in the official evaluation environment?

Since late submissions and re-submissions due to file naming or submission errors will not be accepted, we would like to make sure that our submission can be correctly built and executed by the evaluation system.

A9. Thanks for the detailed questions. Build your executable on RHEL 8 / AlmaLinux 8 / CentOS 8 (glibc 2.28), targeting x86_64. Executable name must be exactly regr_fail_bucketing, as described in the released document. Third party packages (numpy, pandas, sklearn) are fine when bundled inside the PyInstaller binary. Submissions must be self-contained; do not rely on requirements.txt / pip installs / environment.yml since there is no internet access during eval. Bundle everything into the executable, either python or C++. Source (python) is still accepted only as a

fallback if the executable cannot start, but the source path must also be self-contained (refer to the example programs). For C++, submit a statically-linked (or RHEL 8-built) x86_64 executable. Docker is not used.

Q10. I am participating in Problem B.
I would like to clarify the allowed packaging format and evaluation environment for the Alpha Test submission.

According to the Problem B statement, Python programs are recommended to be packed using PyInstaller, and source code is optional.
Since Problem B submissions are submitted via Google Drive, could you please clarify:

1. What is the OS, CPU architecture, Python version, and glibc version of the Problem B evaluation environment?
2. If we submit a PyInstaller-packed executable, should it be built for x86_64 Linux?
3. Are external Python packages such as numpy, scipy, pandas, and scikit-learn allowed if they are bundled inside the executable?
4. If a submitted executable cannot run, will the evaluator try to run the provided source code? If so, are third-party packages installed in the evaluation environment, or should the submission be self-contained?

A10. Build your executable on RHEL 8 / AlmaLinux 8 / CentOS 8 (glibc 2.28), targeting x86_64. Third party packages (numpy, pandas, sklearn) are fine when bundled inside the PyInstaller binary. Bundle everything into the executable. Source is still accepted only as a fallback if the executable cannot start, but the source path must also be self-contained.